

Neural Data Analysis: Encoding

13.1. GOALS OF THIS CHAPTER

The primary goal of this chapter is to introduce you to fundamental methods of analyzing spike trains of single neurons used to characterize their encoding properties: raster plots, peri-stimulus time histograms, and tuning curves. While there are prepackaged tools available for these methods, in this chapter you will program these tools yourself and use them to analyze behavioral data recorded from motor areas of a macaque monkey.

13.2. BACKGROUND

In general, neuroscientists are interesting in knowing what neurons are doing. More specifically, neuroscientists are often interested in neural *encoding*—how neurons represent stimuli from the outside world with changes in their firing properties. Let's say you are studying a neuron from a visual area. You would first present a subject with controlled visual stimuli with a number of known properties—orientation, luminance, contrast, etc. If you are studying a neuron from a motor area, your stimuli might be a set of stereotyped movements which the subject performs. Using standard electrophysiological techniques, you then record the response of the neuron to each stimulus. You can repeat the presentation of a given stimulus and then see how similar (or different) the neuronal responses are. A *raster plot* is a simple method to visually examine the trial-by-trial variability of these responses. You can examine what features these responses have in common by averaging over all responses to create a *peri-stimulus time histogram*. Finally, to capture how the average response of the neuron varies with some sensory or motor feature, you can generate a *tuning curve* that maps the feature value onto the average response of the neuron.

13.3. EXERCISES

13.3.1. Raster Plot

Because action potentials are stereotyped events, the most important information they carry is in their timing. A raster plot replaces each action potential with a tick mark that corresponds to the point where the raw voltage trace crosses some threshold.

Load the dataset for this chapter from the companion website. Contained within that dataset is a variable *spike*, which contains the firing times (in seconds) of a single neuron for 47 trials of the same behavioral task. Here, you are recording from a cell in the motor cortex, and the task involves moving the hand from the same starting position to the same ending position. For each trial, the spike times are centered so that the start of movement coincides with a timestamp of 0 seconds. Because the neuron did not fire the same number of times for each trial, the data are stored in a struct, which is a data structure that can bundle vectors (or matrices) of different lengths. To access the spike times for the first and second trials, type:

```
t1 = spike(1).times;
t2 = spike(2).times;
```

If you look at the workspace, you can verify that the vectors *t1* and *t2* are not the same length. Now plot the first trial as a raster:

```
figure                                %Create a new figure
hold on                              %Allow multiple plots on the same graph
for i = 1:length(t1)                 %Loop through each spike time
    line([t1(i) t1(i)], [0 1])      %Create a tick mark at x = t1(i) with a height of 1
end
ylim([0 5])                          %Reformat y-axis for legibility
xlabel('Time (sec)')                 %Label x-axis
```

Even when you're looking at one trial, it appears that the neuron fires sparsely at first but then ramps up its firing rate a few hundred milliseconds before the start of movement. Now plot the next trial:

```
for i = 1:length(t2)
    line([t2(i) t2(i)], [1 2])
end
```

Your result should look like those in [Figure 13.1](#).

The relationship between the firing rate and start of movement is not nearly as clear in the second trial as in the first trial. To resolve this discrepancy, you can write a loop to plot rasters for all trials. You will do this as part of this chapter's project. However, it would also be nice to know what the response of the "average trial" looks like.

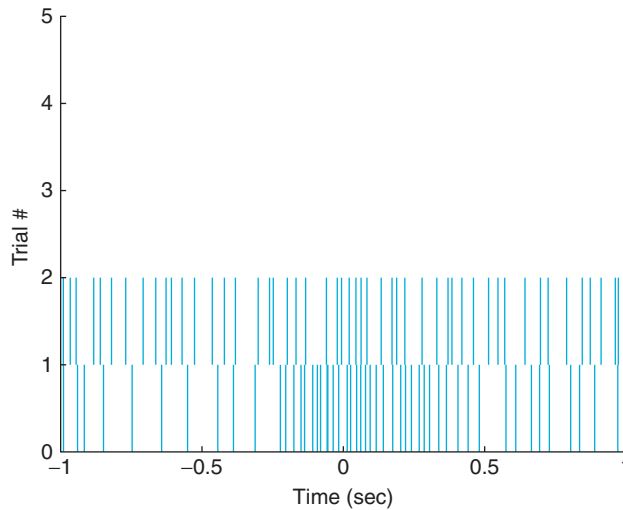


FIGURE 13.1 A raster plot of spike times of the sample neuron for the first two trials.

13.3.2. Peri-Stimulus Time Histogram

The average neural response is captured by the *peri-stimulus time histogram*, which is abbreviated PSTH and sometimes referred to as the *peri-event time histogram*. *Peri-stimulus* means that all the trials are centered relative to some relevant stimulus or event—in this case, the start of movement. *Time histogram* means you divide the time period into a series of bins (0 to 100 ms, 100 to 200 ms, etc.) and count how many spikes fall in each bin for all trials. Luckily, the MATLAB® software has a function that makes this task easy: **histc**. To look at all trials, you will initialize the PSTH with zeros and then sequentially add each trial's results. Try the following:

```
edges = [-1:0.1:1];    %Define the edges of the histogram
psth = zeros(21,1);    %Initialize the PSTH with zeros
for j=1:47              %Loop over all trials
    %Add current trial's spike times
    psth = psth+histc(spike(j).times,edges);
end
bar(edges,psth);        %Plot PSTH as a bar graph
xlim([-1.1 1])         %Set limits of X-axis
xlabel('Time (sec)')    %Label x-axis
ylabel('# of spikes')   %Label y-axis
```

Your results should look like those in [Figure 13.2](#).

Now the pattern in neuronal activity is clear: the firing rate begins to increase about half a second before movement start and returns to baseline by half a second after movement start. Of course, for the y-axis to indicate firing rate in spikes per second, you need to divide each bin's spike count by both the bin width and the number of trials.

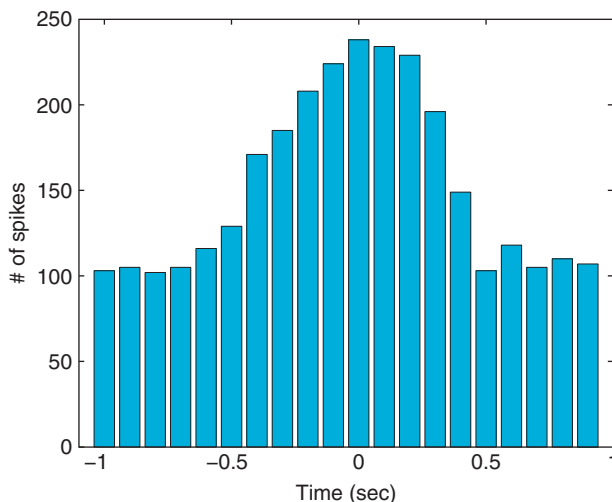


FIGURE 13.2 A peri-stimulus time histogram centered on the start of movement.

13.3.3. Tuning Curves

Many neurons respond preferentially to particular values of the stimulus. Typically, this activity gradually falls off from a maximum (corresponding to the preferred stimulus) along some stimulus dimension (e.g., orientation, direction). By plotting the stimulus dimension on the x-axis and the neural activity (typically a firing rate) on the y-axis, you can determine the preferred stimulus of a neuron. Figure 13.3 shows a tuning curve of a

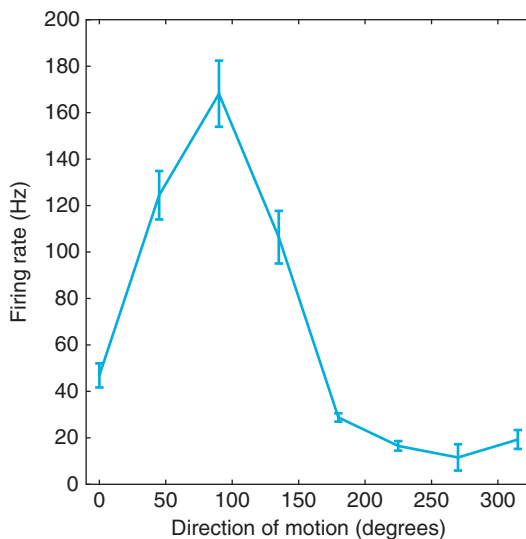


FIGURE 13.3 A tuning curve for a neuron from area MT.

neuron from area MT, which is a part of the visual cortex that aids in the perception of motion. As you can tell, the neuron prefers upward motion (motion toward 90°).

13.3.4. Curve Fitting

Typically, tuning curves like this are fit to a function such as a Gaussian curve or a cosine function. Because all measurements made in the real world come with errors, it is usually impossible to describe empirical data with a perfect functional relationship. Instead, you fit data with a curve that represents a model of the proposed relationship. If this curve fits the data well, then you conclude that your model is a good one.

The simplest relationship you will typically look for is a linear one. Many neurons are thought to encode stimuli linearly. For example, ganglion cells in the limulus (horseshoe crab) increase their firing rate linearly with luminance of a visual stimulus (Hartline, 1940). You can simulate this relationship as follows:

```
x = 1:20;           %Create a vector with 20 elements
y = x;              %Make y the same as x
z = randn(1,20);    %Create a vector of random numbers with same dimensions as x
y = y + z;           %Add z to y, introducing random variation
plot(x,y, 'o')       %Plot the data as a scatter plot
xlabel('Luminance')
ylabel('Firing rate')
```

MATLAB contains prepackaged tools for fitting linear relationships. Just click on the figure, select Tools, and then select Basic Fitting. Check the boxes for Linear and Show equations, and you will see the line and equation that best fit your data. However, you might also like to be able to do this yourself. The command in MATLAB to fit data to a polynomial is **polyfit**. For example:

```
p=polyfit(x,y,1) %fits data to a linear, 1st degree polynomial
```

The first value in p is the slope and the second value is the y-intercept. If you plot this fitted line, your result should be similar to [Figure 13.4](#).

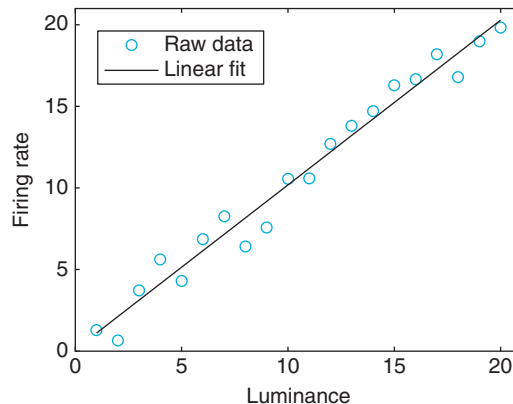


FIGURE 13.4 A linear fit of the relationship between the firing rate of a simulated ganglion cell and the luminance of the stimulus.

```
hold on           %Allow 2 plots of the same graph
yFit = x*p(1)+p(2); %Calculate fitted regression line
plot(x,yFit)      %Plot regression
```

Now you will fit data to a more complicated function—a cosine. First, generate some new simulated data:

```
x = 0 : 0.1 : 30; %Create a vector from 0 to 10 in steps of 0.1
y = cos (x);      %Take the cosine of x, put it into y
z = randn(1,301); %Create random numbers, put it into 301 columns
y = y + z;        %Add the noise in z to y
figure           %Create a new figure
plot (x,y)        %Plot it
```

MATLAB does not have a built-in function for fitting this to a cosine, but it does have a nonlinear curve-fitting function: **nlinfit**. You will need to specify the details of the fit. Here, you will use a cosine function with the y-offset, amplitude, and phase as free parameters. You can define this function “inline,” which means it can be used by other functions in MATLAB in the same session or M-file.

Type this command to define a generic cosine function:

```
mystring = 'p(1) + p(2) * cos ( theta - p(3) )'; %Cosine function in string form
```

Here, $p(1)$ represents the y-offset; $p(2)$, the amplitude; and $p(3)$, the phase. You can assume the frequency is 1. Now enter the following:

```
myfun = inline ( mystring, 'p', 'theta' ); %Converts string to a function
```

This function accepts angles *theta* and parameter vector *p* and transforms them using the relationship stored in *mystring*.

```
p = nlinfit(x, y, myfun, [1 1 0] ); %Least squares curve fit to inline function "myfun"
```

The first parameter of **nlinfit** is a vector of the x-values (the angle *theta* in radians). The second parameter is the observed y-values. The third parameter is the name of the function to fit, and the last parameter is a vector with initial guesses for the three free parameters of the cosine function. If the function doesn't converge, use a different initial guess. The **nlinfit** function returns the optimal values of the free parameters (stored in *p*) that fit the data with the cosine function, as determined by a least squares algorithm.

Optionally, instead of defining a function inline, you can also save a function in an M-file. In that case, you will need to include an @ (at) symbol before the function name, which will allow MATLAB to access the function as if it were defined inline:

```
p = nlinfit(x, y, @myfun, [1 1 0] ); %Least squares curve fit to function "myfun.m"
```

You can use the inline function to convert the optimized parameters into the fitted curve. After plotting this, your result should look similar to [Figure 13.5](#).

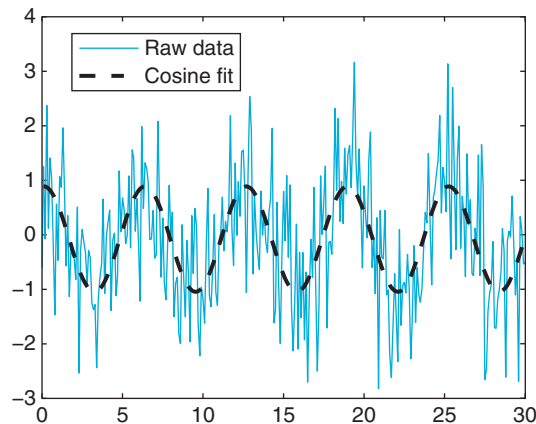


FIGURE 13.5 A nonlinear fit of a simulated, noisy cosine relationship.

```
hold on           %allows 2 plots of the same graph
yFit = myfun(p,x); %calculates fitted regression line
plot(x,yFit,'k')  %plots regression
```

13.4. PROJECT

The data that you will use for your project were recorded from the primary motor cortex (abbreviated MI) of a macaque monkey (data courtesy of the Hatsopoulos laboratory). MI is so named because movements can be elicited by stimulating this area with a small amount of electricity. It has also been shown that MI has direct connections to the spinal cord. MI plays an important role in the control of voluntary movement (as opposed to reflexive movements). This doesn't mean that MI directly controls movement, because other areas in the basal ganglia and the brainstem are important as well. Animals with a lesioned MI can still make voluntary movements, albeit less dexterously than before. However, it is clear that MI contains information about voluntary movement, usually a few hundred milliseconds before it actually happens. Thus, the usual terminology of "stimulus" and "response" is awkward here because the neural "response" usually precedes the experimental "stimulus." There is also a somatotopic map in MI, meaning that there are separate areas corresponding to face, arm, leg, or hand movements. These data are recorded from the arm area.

The behavioral data were collected using a manipulandum, which is an exoskeleton that fits over the arm and constrains movement to a 2-D plane. Think of the manipulandum as a joystick controlled with the whole arm. The behavioral task was the center-out paradigm pioneered by Georgopoulos and colleagues (1982). The subject first holds the cursor over the center target for 500 ms. Then a peripheral target appears at one of eight locations arranged in a circle around the center target. In this task there is an instructed delay, which means that after the peripheral target appears, the subject must wait 1000–1500 ms for a go cue. After the go cue, the subject moves to and holds on the peripheral target for 500 ms, and the trial is completed.

There are two interesting time windows here. Obviously, MI neurons should respond during a time window centered around the go cue, since this is when voluntary movement begins. However, MI neurons also respond during the instructed delay. This result is somewhat surprising because the subject is holding still during this time. The usual interpretation is that the subject is imagining or preparing for movement to the upcoming target. This means that MI is involved in planning as well as executing movement.

If you treat the direction to the peripheral target as the stimulus, you can arrange the neuronal responses in a tuning curve. These can be described with the same cosine curve used before, where the phase of the fitted cosine corresponds to the preferred direction of the neuron.

In this dataset, the neuronal spiking is stored in a struct called *unit*. Information for unit #1 is accessed with *unit(1)*. Spike times are stored in *unit(1).times*. There are three more important variables: the instruction cue times are stored in *instruction*, the go cue times are stored in *go*, and the direction of peripheral target is stored in *direction* (1 corresponds to 0 degrees, 2 corresponds to 45 degrees, etc.).

In this project, you are asked to do the following:

1. Make raster plots and PSTHs for all the neurons for both time periods: instruction cue to 1 second afterward, and 500 ms before the movement onset to 500 ms afterward. Which neurons are the most responsive? Print out a few examples. Do you think the PSTHs are a good summary of the raster plots? How does the time course of the responses differ between the two time periods?
2. Create tuning curves and fit a cosine tuning curve to the firing rates of all neurons for each time period. Report the parameters of the fit for each neuron and save this information for later chapters. How good of a description do you think the cosine curve is? Do the tuning curves differ between the two time periods? If so, why do you think this is?

Figures 13.6 and 13.7 show examples of what your results might look like. The locations of the smaller plots correspond to the locations of their associated peripheral targets. Here,

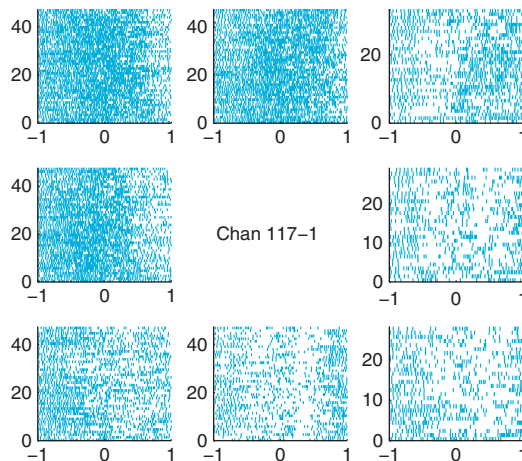


FIGURE 13.6 An example of a full raster plot for the first neuronal unit recorded from electrode #117.

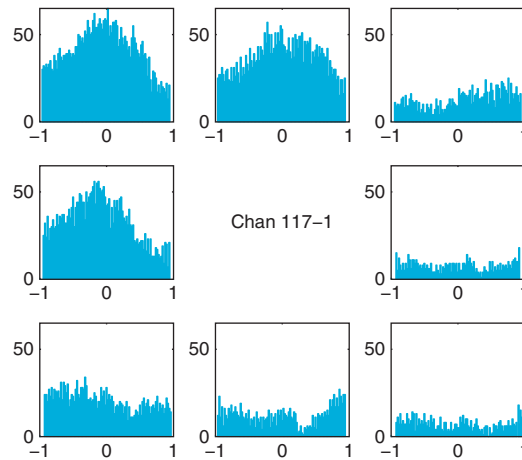


FIGURE 13.7 An example of a full peri-stimulus time histogram for the first neuronal unit recorded from electrode #117.

a timestamp of 0 seconds corresponds to the start of movement. You can use the command **subplot** to subdivide the plotting area. For example, the command **subplot(3,3,i)** makes the i^{th} square in a 3×3 grid the active plotting area.

MATLAB FUNCTIONS, COMMANDS, AND OPERATORS COVERED IN THIS CHAPTER

histc
randn
bar
polyfit
nlinfit
subplot